

PSProviders, PSDrives et Manipulation du Registre Windows

Ce document détaille la couche d'abstraction des fournisseurs de données PowerShell (PSProviders), le mécanisme des lecteurs virtuels (PSDrives), et la méthodologie rigoureuse de manipulation de la base de registre Windows sans interface graphique.

1. La Couche d'Abstraction : Les Fournisseurs PowerShell (PSProviders)

1.1 Qu'est-ce qu'un PSProvider ?

Dans la majorité des systèmes d'exploitation, chaque magasin de données dispose de ses propres outils et syntaxes de manipulation :

- Les fichiers et dossiers se gèrent via des commandes de disque (dir, cd, copy).
- La base de registre nécessite l'outil graphique regedit.exe ou l'utilitaire en ligne de commande reg.exe.
- Les certificats numériques s'administrent via la console MMC certlm.msc.
- Les variables d'environnement requièrent des commandes spécifiques (set).

PowerShell élimine ce cloisonnement grâce à une couche d'abstraction appelée **PSProvider** (*PowerShell Provider* ou Fournisseur PowerShell).

Un PSProvider est un adaptateur logiciel interne (.NET) qui convertit un magasin de données complexe en une structure arborescente standardisée. Grâce à cette passerelle, PowerShell vous permet d'interagir avec la base de registre ou le magasin de certificats **exactement comme s'il s'agissait de dossiers et de fichiers sur un disque dur**.

1.2 Cartographie des PSProviders du système

La commande Get-PSProvider permet de recenser les fournisseurs intégrés à votre environnement :

Get-PSProvider

Le tableau de sortie expose plusieurs colonnes techniques fondamentales :

Colonne	Rôle opérationnel
Name	Le nom d'identification du fournisseur (ex : Registry, FileSystem, Certificate, Environment).
Capabilities	Les fonctionnalités avancées supportées (ex : ShouldProcess pour -WhatIf, Filter pour filtrer à la source, Transactions).
Drives	Les points de montage (lecteurs virtuels) actuellement rattachés à ce fournisseur.

Les quatre grands fournisseurs universels pour l'administrateur système sont :

1. **FileSystem** : gère les partitions locales et partages réseau (C:, D:, \\Serveur\Partage).
2. **Registry** : expose les ruches du Registre Windows (HKLM:, HKCU:).
3. **Certificate** : donne accès aux magasins de certificats numériques de la machine et de l'utilisateur (Cert:\LocalMachine, Cert:\CurrentUser).
4. **Environment** : donne accès aux variables d'environnement de la session active (Env:).

2. Les Lecteurs Virtuels : Le Mécanisme des PSDrives

2.1 Distinction entre lecteur physique et PSDrive

Un **PSDrive** (*PowerShell Drive*) est un point de montage logique associé à un PSProvider.

- Dans l'explorateur de fichiers classique, une lettre de lecteur (C:) désigne exclusivement une partition physique ou un volume logique formaté en NTFS/FAT32.
- Dans PowerShell, un PSDrive désigne **n'importe quel magasin de données** rattaché à un fournisseur, qu'il soit physique, mémoire ou système.

Pour inventorier l'ensemble des lecteurs actifs :

Get-PSDrive

Vous y observerez des lecteurs non physiques comme :

- Env : liste toutes les variables d'environnement sous forme d'éléments accessibles.
- HKLM : pointe directement sur la ruche *HKEY_LOCAL_MACHINE* du registre.
- HKCU : pointe sur la ruche *HKEY_CURRENT_USER*.
- Cert : pointe sur l'arborescence des certificats Windows.

- WSMAN : permet de configurer le moteur d'écoute WinRM.

2.2 Navigation universelle dans les PSDrives

Puisque ces magasins adoptent le comportement d'un système de fichiers, les commandes de base fonctionnent de manière uniforme :

```
# Consulter les variables d'environnement  
Get-ChildItem -Path Env:
```

```
# Lire la valeur d'une variable précise  
(Get-Item -Path Env:\COMPUTERNAME).Value
```

```
# Parcourir les magasins de certificats  
Get-ChildItem -Path Cert:\LocalMachine
```

2.3 Création d'un PSDrive personnalisé (New-PSDrive)

Dans l'administration quotidienne, les chemins système sont souvent très longs et propices aux erreurs de frappe (ex: HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System).

La cmdlet New-PSDrive permet de créer un **lecteur raccourci** temporaire vers une sous-branche ciblée :

```
New-PSDrive -Name "SecPol" -PSProvider "Registry" -Root  
"HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"
```

- **Paramètres obligatoires :**
 - -Name : le nom court attribué au nouveau lecteur (sans deux-points).
 - -PSProvider : le fournisseur gestionnaire (Registry, FileSystem, etc.).
 - -Root : le chemin source ciblé qui fera office de racine pour ce nouveau lecteur.

Une fois monté, vous pouvez y accéder directement :

```
# Interrogation immédiate de la branche cible via le raccourci  
Get-ItemProperty -Path "SecPol:"
```

Règle de persistance : Par défaut, un PSDrive créé via New-PSDrive n'existe que dans l'espace mémoire de la console ou du script qui l'a instancié. Dès la fermeture de la session PowerShell, le lecteur est détruit automatiquement. Si vous souhaitez le supprimer manuellement avant la

fin de session, utilisez `Remove-PSDrive -Name "SecPol"`.

3. La Base de Registre dans PowerShell : (Item vs ItemProperty)

3.1 Anatomie du Registre Windows

Dans `regedit.exe`, l'architecture se divise en deux éléments distincts :

1. **Les Clés (et Sous-clés)** : Les conteneurs jaunes disposés en arborescence de dossiers.
2. **Les Valeurs** : Les entrées de données situées dans le panneau de droite à l'intérieur d'une clé (ex : une valeur de type chaîne `REG_SZ` ou numérique `REG_DWORD`).

[HKLM:\SOFTWARE\MonApplication] <--- C'est une CLÉ (Conteneur)

```
|
|--- CodeSite = "PARIS-01" <--- C'est une VALEUR (Propriété)
|--- PortEcoute = 8080 <--- C'est une VALEUR (Propriété)
```

3.2 La transcription dans le modèle objet PowerShell

Dans le système de fichiers :

- Un dossier est un `Item`.
- Un fichier est également un `Item` (situé à l'intérieur du dossier).

Dans la base de registre :

- Une Clé de registre est un `Item`.
- Une Valeur de registre N'EST PAS un fichier : c'est une propriété de la clé, donc une `ItemProperty`.

Élément dans le Registre	Concept PowerShell	Famille de commandes à employer
La Clé (Dossier)	<code>Item</code>	<code>Test-Path</code> , <code>Get-Item</code> , <code>New-Item</code> , <code>Remove-Item</code>
La Valeur (Donnée interne)	<code>ItemProperty</code>	<code>Get-ItemProperty</code> , <code>New-ItemProperty</code> , <code>Set-ItemProperty</code> , <code>Remove-ItemProperty</code>

Si vous tentez de créer une valeur de registre avec `New-Item`, PowerShell va tenter de créer une sous-clé (un sous-dossier), ce qui génère une structure invalide. Pour écrire un paramètre

dans une clé, vous devez obligatoirement employer New-ItemProperty ou Set-ItemProperty.

4. Typologie des Données du Registre (PropertyType)

Dans la base de registre, chaque valeur possède un type technique strict. Lors de la création d'une nouvelle valeur via New-ItemProperty, il est impératif de déclarer le type adéquat via le paramètre -PropertyType :

Type PowerShell (-PropertyType)	Type natif Windows (regedit)	Cas d'usage en entreprise
String	REG_SZ	Chaîne de texte simple (chemins, noms de serveurs, identifiants).
DWord	REG_DWORD	Entier 32 bits (indicateurs binaires 0 ou 1, numéros de ports).
QWord	REG_QWORD	Entier 64 bits (grandes grandeurs mémoires, compteurs système).
ExpandString	REG_EXPAND_SZ	Chaîne contenant des variables d'environnement non résolues (ex : %SystemRoot%).
MultiString	REG_MULTI_SZ	Tableau de chaînes textuelles multiples (listes d'exclusions, cibles).
Binary	REG_BINARY	Données binaires brutes au format hexadécimal.

5. Guide Opérationnel des Commandes du Registre

5.1 Vérifier la présence d'une clé (Test-Path)

Avant toute lecture ou écriture, le script doit vérifier que l'arborescence parente existe :

```
$Cle = "HKLM:\SOFTWARE\EntrepriseConfig"
```

```
if (Test-Path -Path $Cle) {  
    Write-Host "La cle existe." -ForegroundColor Green  
} else {  
    Write-Host "La cle est absente." -ForegroundColor Yellow  
}
```

5.2 Créer une clé de registre (New-Item)

Pour créer une nouvelle arborescence de configuration :

Le parametre -ItemType Directory est recommande pour expliciter la creation de conteneur
New-Item -Path "HKLM:\SOFTWARE\EntrepriseConfig" -ItemType Directory -Force

Le paramètre -Force permet de créer la clé sans générer d'erreur bloquante si celle-ci existe déjà.

5.3 Créer une valeur de registre (New-ItemProperty)

Une fois la clé présente, on injecte les entrées de configuration :

1. Creation d'une valeur texte (String)

```
New-ItemProperty -Path "HKLM:\SOFTWARE\EntrepriseConfig" `  
    -Name "CodeSite" `  
    -Value "PARIS-01" `  
    -PropertyType String `  
    -Force
```

2. Creation d'un drapeau numerique (DWord)

```
New-ItemProperty -Path "HKLM:\SOFTWARE\EntrepriseConfig" `  
    -Name "AuditActif" `  
    -Value 1 `  
    -PropertyType DWord `  
    -Force
```

5.4 Lire les valeurs d'une clé (Get-ItemProperty)

Pour extraire les paramètres d'une clé de registre :

Lecture de toutes les valeurs sous forme d'objet

```
$Config = Get-ItemProperty -Path "HKLM:\SOFTWARE\EntrepriseConfig"
```

Extraction de la propriété souhaitée par notation pointée

```
$Config.CodeSite
```

```
$Config.AuditActif
```

Note moderne (PowerShell 5.0+) : La cmdlet `Get-ItemPropertyValue -Path "HKLM:\SOFTWARE\EntrepriseConfig" -Name "CodeSite"` permet d'extraire directement la valeur brute sans passer par l'objet conteneur.

5.5 Modifier une valeur existante (Set-ItemProperty)

Lorsque la valeur existe déjà et qu'il faut simplement ajuster sa donnée :

```
Set-ItemProperty -Path "HKLM:\SOFTWARE\EntrepriseConfig" -Name "CodeSite" -Value "LYON-02"
```

5.6 Supprimer des entrées (Remove-ItemProperty vs Remove-Item)

- **Supprimer une valeur unique sans toucher à la clé :**
`Remove-ItemProperty -Path "HKLM:\SOFTWARE\EntrepriseConfig" -Name "AuditActif"`
- **Supprimer la clé entière et toutes les sous-branches qu'elle contient :**
`Remove-Item -Path "HKLM:\SOFTWARE\EntrepriseConfig" -Recurse -Force`

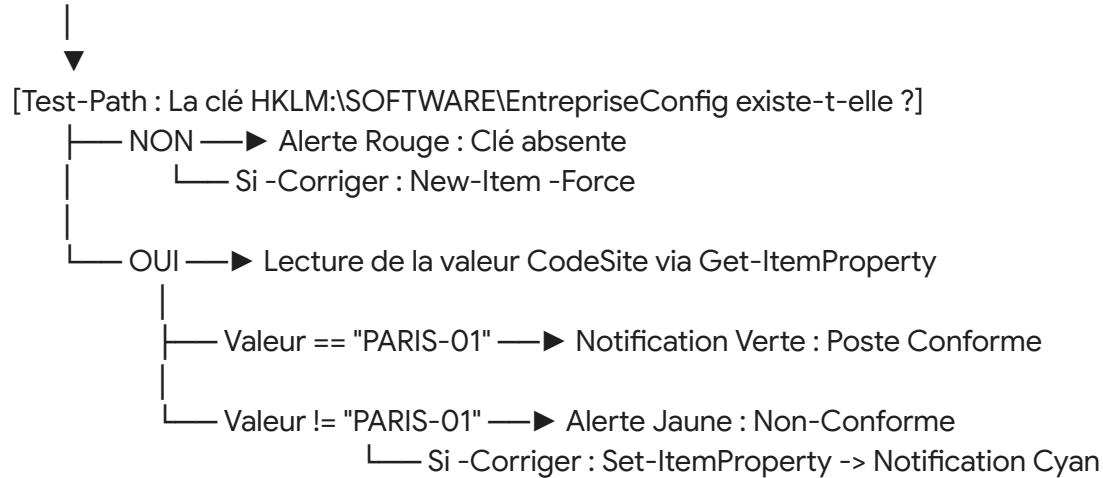
6. Méthodologie d'un Script d'Audit et de Remédiation

En ingénierie de production, un script ne doit jamais modifier des paramètres système à l'aveugle. Il applique un principe d'**idempotence** :

1. **Étape 1 : Constat d'état (Audit) :** Le script lit le registre et compare l'état réel de la machine avec la politique de conformité attendue.
2. **Étape 2 : Décision de remédiation :**
 - Si la machine est conforme : le script n'effectue aucune action modificatrice.
 - Si une dérive est détectée : il affiche un avertissement clair.
 - Si l'opérateur a spécifié un commutateur de correction (`-Corriger`), le script applique immédiatement la valeur attendue pour rétablir la conformité.

Modèle de logique algorithmique attendu :

[Lancement du Script avec -SiteAttendu "PARIS-01"]



7. Sécurité et Bonnes Pratiques

1. **Élévation de privilèges obligatoire** : La ruche HKLM: (*HKEY_LOCAL_MACHINE*) contient la configuration globale de la machine Windows. Toute tentative d'écriture sans console exécutée en mode administrateur renverra une exception d'accès refusé (*UnauthorizedAccessException*).
2. **Ne jamais toucher aux branches système sans sauvegarde** : La ruche HKLM:\SYSTEM ou HKLM:\SAM ne doit jamais être modifiée arbitrairement. Les scripts de configuration d'entreprise doivent cantonner leurs paramètres personnalisés sous HKLM:\SOFTWARE\<NomEntreprise>.
3. **Privilégier Set-ItemProperty à New-ItemProperty -Force** : Set-ItemProperty modifie la donnée sans recréer le type de propriété, réduisant les risques d'altération de métadonnées.